

DataMining LINFO2275: Markov Decision Processes

Group 9

Berlaimont Gaetan (DATE) -

Henrion Benoit (MAP) -

Zeamari Yassine (DATS) -

March 30, 2025

1 Introduction

Markov Decision Processes (MDPs) are a way to model decision-making when there's some uncertainty about what happens next. They're used in areas like game strategy, robotics, and artificial intelligence to help figure out the best choices over time.

One powerful method for solving MDPs is the Value Iteration Algorithm. This algorithm works by repeatedly updating the value of each possible states until it finds the best possible strategy. It does this using a simple rule (called the Bellman equation) that helps estimate long-term rewards or cost for different choices.

In this report, we'll aims at applying the value iteration algorithm solving Markov decision processes in the framework of a Snakes and Ladders game.

2 Theoretical explanation

In a MDPs we have a set of states $S = \{1, 2, \dots, n\}$. In each states $s = k$, we have a set of admissible actions, $a \in U(k)$.

When we choose an actions $a_t = u(s_t)$ at time t in state s_t :

1. a bounded cost $0 \leq c(u(s_t)|s_t) \leq \infty$ is incurred.
2. the system jumps to state s_{t+1} with a probability mass $P(s_{t+1} = k' | s_t = k, u(s_t) = a_t) = p(k'|k, a)$.

which only depends on the current state k and not on the past states.

The goal of MDPs is to learn a policy $\pi = \{u(1), u(2), \dots, u(n)\}$ that maps each observed state k on a unique action a chosen among the available action in state k , $U(k)$. In other word, the goal is to reach a destination state $s = d$, from some initial state $s_0 = k_0$ at time $t = 0$, while minimizing the expected cost

$$V_\pi(s_0 = k_0) = \mathbb{E}_{s_1, s_2, \dots} \left[\sum_{t=0}^{\infty} c(u(s_t)|s_t) | s_0 = k_0, \pi \right]$$

Once we have reached the destination state we stay there forever with no cost ($p(d|d, i) = 1$ and $c(d|d) = 0$). Thus an optimal policy is given by

$$V^*(k_0) = \min_{\pi} \{V_\pi(k_0)\}$$

To compute the expected cost for each states we can derive the Bellman's equation of optimality.

$$\begin{aligned}
V_\pi(k_0) &= \mathbb{E}_{s_1, s_2, \dots} \left[\sum_{t=0}^{\infty} c(u(s_t)|s_t) \middle| s_0 = k_0, \pi \right] \\
&= c(u(k_0)|k_0) + \mathbb{E}_{s_1, s_2, \dots} \left[\sum_{t=1}^{\infty} c(u(s_t)|s_t) \middle| s_0 = k_0, \pi \right] \\
&= c(u(k_0)|k_0) + \mathbb{E}_{s_1} \left[\mathbb{E}_{s_2, s_3, \dots} \left[\sum_{t=1}^{\infty} c(u(s_t)|s_t) \middle| s_1, s_0 = k_0, \pi \right] \middle| s_0 = k_0, \pi \right] \\
&= c(u(k_0)|k_0) + \mathbb{E}_{s_1} [V_\pi(s_1) | s_0 = k_0, \pi] \\
&= c(u(k_0)|k_0) + \sum_{k_1=1}^n P(s_1 = k_1 | s_0 = k_0, u(k_0)) V_\pi(k_1)
\end{aligned}$$

after simplification we obtain,

$$V_\pi(k_0) = c(a_0|k_0) + \sum_{k_1=1}^n p(k_1|k_0, a_0) V_\pi(k_1)$$

Thus,

$$\begin{aligned}
V^*(k_0) &= \min_{\pi} V_\pi(k_0) = \min_{(a_0, a_1, \dots)} V_\pi(k_0) \\
&= \min_{a_0 \in U(k_0)} \left\{ c(a_0|k_0) + \sum_{k_1=1}^n p(k_1|k_0, a_0) V^*(k_1) \right\}
\end{aligned}$$

Which yield the value-iteration algorithm,

$$\begin{cases} \hat{V}(k) \leftarrow \min_{a \in U(k)} \left\{ c(a|k) + \sum_{k'=1}^n p(k'|k, a) \hat{V}(k') \right\}, k \neq d \\ \hat{V}(d) \leftarrow 0, \text{ where } d \text{ is the destination state} \end{cases}$$

This update the expected cost for reaching the destination from each state until convergence. It takes for each current state k the cost of the chosen action a plus, for each reachable states from applying action a the expected cost for reaching the destination, weighted by their probability distribution. After convergence the best action in each states k can be retrieved by

$$\arg \min_{a \in U(k)} \left\{ c(a|k) + \sum_{k'} p(k'|k, a) \hat{V}(k') \right\}$$

3 Implementation

We implemented the value iteration algorithm to compute the expected cost for each states:

Algorithm 1 pseudo-code of value-iteration

Require: layout $\in \{0, 1, 2, 3\}^{1 \times 15}$, circle $\in \mathbb{B}$

$P \leftarrow$ Compute transition probability

Expec $\leftarrow 0^{1 \times 15}$

while $\delta < 1e^{-6}$ **do**

for each $k \in \text{state}$ **do**

 Expec[k] $\leftarrow$ Best $V_k(\text{layout}, P, \text{action}, \text{Expec})$

end for

$\delta \leftarrow$ Biggest entry change of Expec

end while

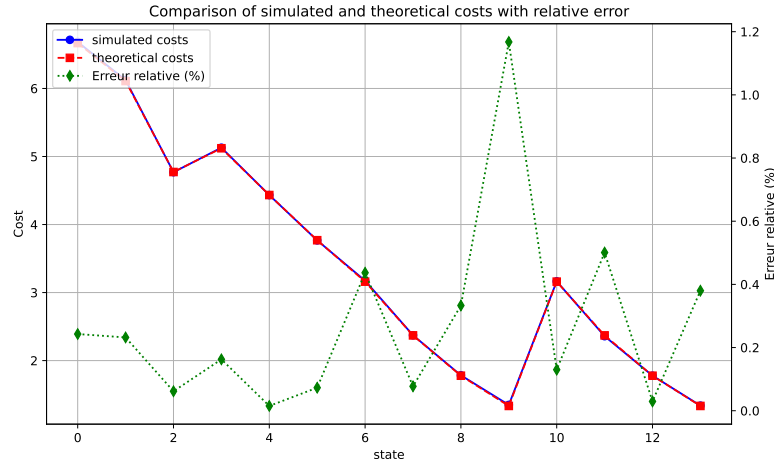
Where V_k here Compute the expected cost for a given state k and a given action a , for one iteration of the value iteration algorithm. Then, we retrieve, at each states, the best action that give the least expected cost.

4 Comparisons between theoretical and empirical results

This section examines the influence of different trap placements on the policy. It includes a comparison of theoretical predictions with empirical results (repeated 10000 times) generated using the optimal policy obtained through the value iteration algorithm. Finally, the performance of the optimal policy is assessed and compared with that of sub-optimal policies.

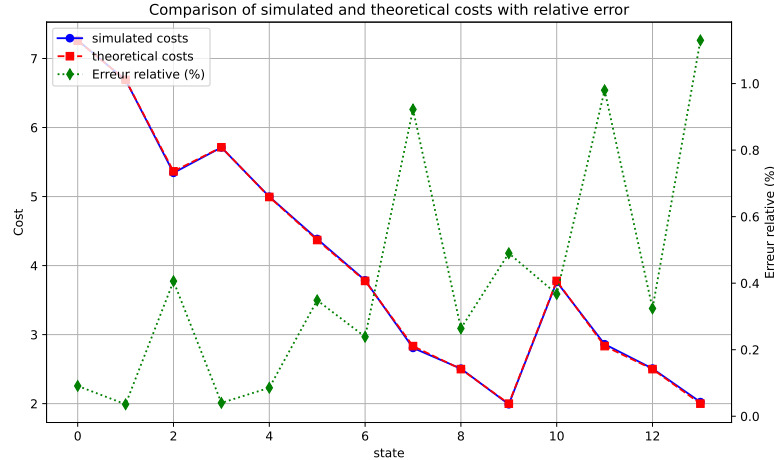
4.1 Layout 1

For the first layout, we chose the most basic layout: no traps and no circular path (circle=False).



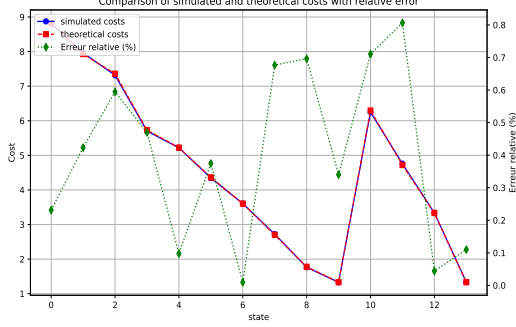
4.2 Layout 2

For the second layout, we kept the same structure as the first layout, but this time, we included the circular path (circle=True).

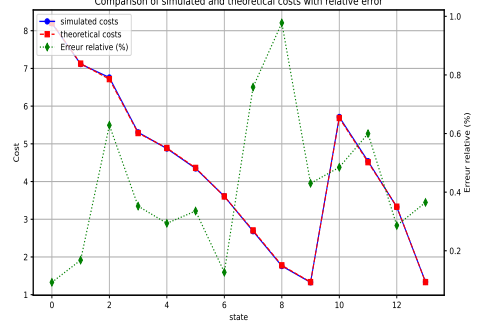


4.3 Layout 3

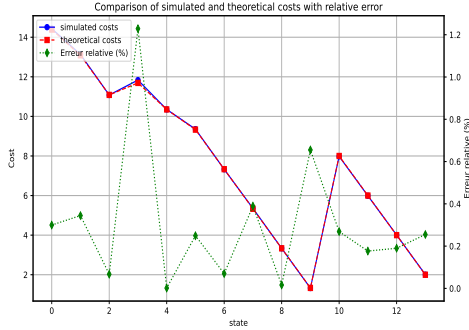
To analyze the impact of traps, we progressively increase the number of traps by replacing three zero-value positions with traps (and no circular path, circle = False). This allows us to compare theoretical and simulated costs across different levels of difficulty.



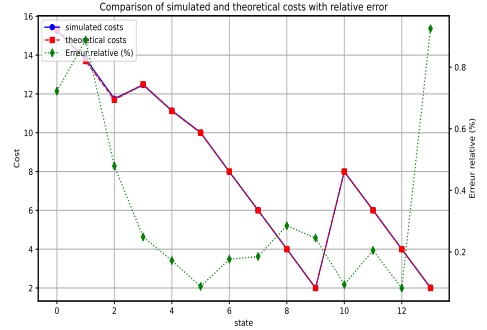
(a) layout : [0,0,2,0,0,0,0,3,0,0,0,0,1,0,0]



(b) Layout : [0,0,2,0,4,0,0,3,0,0,4,0,1,0,0]



(c) Layout : [0,0,2,0,4,1,0,3,1,0,4,0,1,2,0]



(d) Layout : [0,3,2,0,4,1,0,3,1,1,4,3,1,2,0]

As the number of traps increases, we see an increase in the total cost, which makes sense since traps make the environment more difficult to traverse.

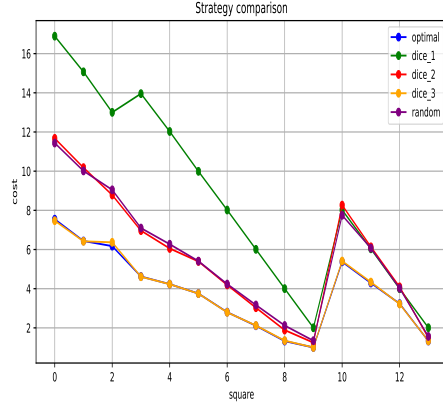
We note that the theoretical and simulated costs remain consistent, the error relative costs remains very low, this indicates that the Value Iteration algorithm is robust and capable of accurately estimating optimal costs even under difficult conditions.

5 Strategy comparison

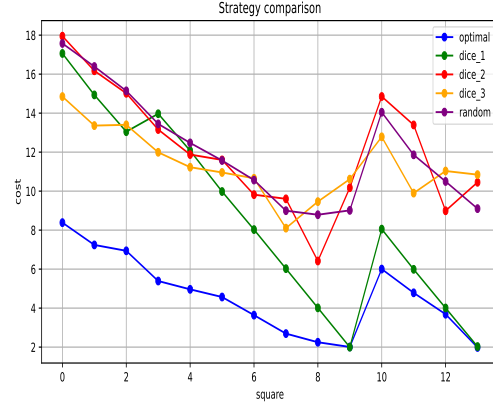
we compare the costs associated with various strategies, including the optimal strategy derived from Markov decision processes and several suboptimal alternatives, highlighting their respective performances across different states.

5.1 case 1

In this analysis, we consider a board without traps to isolate the impact of different strategies. First, we evaluate the strategies on a standard board without circular path. Then, we introduce the circular condition to assess how it influences decision-making



(a) circle = False



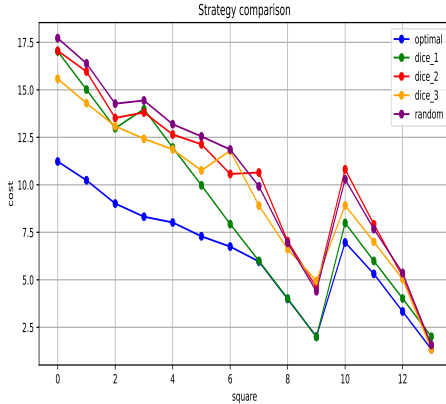
(b) circle = True

In the non-circular case (a), the optimal strategy (blue) and the risky dice strategy (orange) consistently yields the lowest cost, while the other strategies (normal dice, security dice, and random) show higher costs with varying performance.

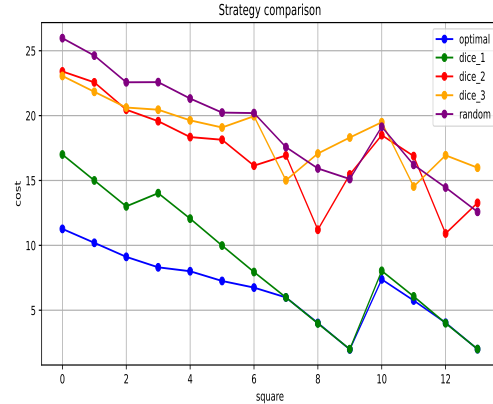
In the circular case (b), costs generally increase for all strategies, particularly for suboptimal ones, and still the optimal strategy the best.

5.2 case 2

In this case, we consider a board configuration with traps placed at specific positions: $[0,0,2,0,4,0,0,3,0,1,4,0,1,0,0]$ reveals significant differences between Circle=False and Circle=True modes.



(a) circle = False, optimal dice strategy is: $[3, 3, 2, 3, 3, 3, 2, 1, 1, 1, 3, 1, 1, 3]$



(b) circle = True, optimal dice strategy is: $[3, 3, 2, 3, 3, 3, 2, 1, 1, 1, 3, 3, 1, 1]$

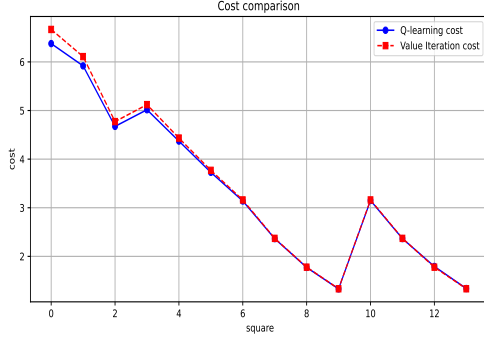
In non-circular mode, costs are generally lower with faster convergence to low cost, while circular mode imposes higher costs. The optimal strategy consistently outperforms other approaches. The others strategies show varying performance, with random dice generally poor, and security dice sometimes close to optimal.

6 Q-learning reinforcement learning

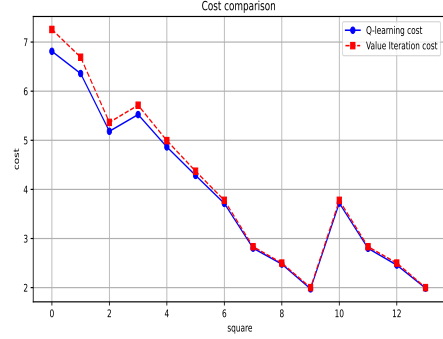
Q-Learning is a fundamental algorithm in reinforcement learning that enables an agent to learn optimal actions through interaction with an environment. By iteratively exploring the environment and updating the Q-values using the formula: $Q(k, a) \leftarrow Q(k, a) + \alpha(t) [c(a|k) + \min_{a'} Q(k', a') - Q(k, a)]$.

We can use a stochastic approximation approach to dynamically adapt the learning rate $\alpha(t)$. A classic method is: $\frac{1}{1+N(k,a)}$, where $N(k,a)$ is the number of times the action state (k,a) has been visited.

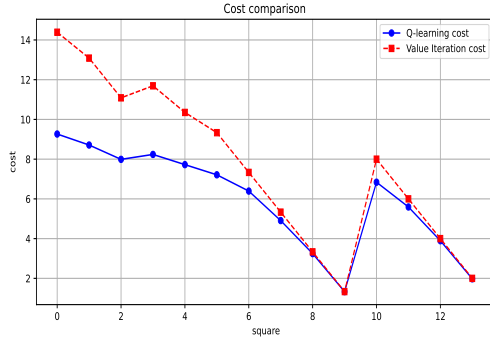
We will compare the performance of the strategies obtained by Q-Learning and Value Iteration, evaluating their convergence and effectiveness



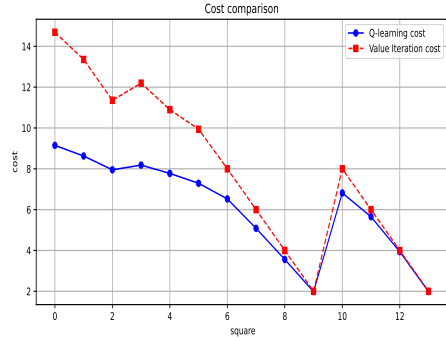
(a) basic layout: no traps and no circular path



(b) no traps and with circular path



(c) layout=[0,0,2,0,4,1,0,3,1,0,4,0,1,2,0] and no circular path



(d) layout=[0,0,2,0,4,1,0,3,1,0,4,0,1,2,0] and with circular path

In the case (a) and (b) both Q-learning and Value Iteration yield very similar cost values, which suggests that in a deterministic and simple environment.

With traps, (c) and (d), a difference starts to emerge. Q-learning appears to achieve a lower cost.

In more complex environments (with traps), Q-learning can find alternative strategies that are better suited, while Value Iteration strictly follows computed state values.

7 conclusion

This report explored the use of Markov Decision Processes (MDPs) and Value Iteration to optimize decision-making in the game of Snakes and Ladders. By comparing with suboptimal strategies, our results showed that the algorithm effectively minimized the expected cost of reaching the destination.

Finally, we compared Q-learning and Value Iteration, highlighting that Q-learning can be more efficient in environments with more traps. Since Q-learning learns from experience and does not require a full model of the environment, it adapts better to complex layouts.